Practical Demo: Duration, Convexity, and Portfolio Repricing

Table of contents

Purpose and Assumptions

This demo prices a small bond portfolio from contractual cash flows, measures Macaulay duration, modified duration, DV01/PVBP, and convexity, and compares approximations with exact repricing. All bonds and yields are **illustrative, market-like inputs**, not live quotes. Bonds pay semiannual coupons; yields are nominal annual yields compounded semiannually; settlement is on a coupon date; default, taxes, accrued interest, and embedded options are ignored.

Workflow stage	In this case
Observed on a position report	Face amount, coupon, maturity, and yield
Calculated	Full price, duration, DV01, and convexity

Workflow stage	In this case
Checked	Approximation error against exact cash-flow repricing
Used for	Portfolio shock P&L and identification of curve risk

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

bonds = pd.DataFrame({
    "Bond": ["2-year Treasury", "5-year agency", "10-year corporate"],
    "Face value ($mm)": [12.0, 10.0, 8.0],
    "Coupon rate": [0.040, 0.0475, 0.0575],
    "Maturity (years)": [2, 5, 10],
    "Yield": [0.0410, 0.0485, 0.0610],
})
frequency = 2
```

1 Full Cash-Flow Pricing and Risk Measures

For cash flow CF_k at semiannual period k,

$$P = \sum_{k=1}^{n} \frac{CF_k}{(1 + y/m)^k}.$$

Macaulay duration is the present-value-weighted average payment time. Modified duration is $D_{Mac}/(1 + y/m)$, and

$$DV01 = PD_{mod}(0.0001).$$

```
def bond_metrics(face, coupon_rate, maturity, ytm, frequency=2):
    periods = int(round(maturity * frequency))
    times = np.arange(1, periods + 1) / frequency
    cash_flows = np.full(periods, face * coupon_rate / frequency)
    cash_flows[-1] += face
    period_yield = ytm / frequency
    pv = cash_flows / (1 + period_yield) ** np.arange(1, periods + 1)
```

```
    price = pv.sum()
    macaulay = np.sum(times * pv) / price
    modified = macaulay / (1 + period_yield)
    k = np.arange(1, periods + 1)
    convexity = np.sum(
        cash_flows * k * (k + 1)
        / (frequency**2 * (1 + period_yield) ** (k + 2))
    ) / price
    dv01 = price * modified * 0.0001
    return price, macaulay, modified, dv01, convexity

rows = []
for bond in bonds.itertuples(index=False):
    face = bond[1] * 1_000_000
    metrics = bond_metrics(face, bond[2], bond[3], bond[4], frequency)
    rows.append(metrics)

metrics = pd.DataFrame(rows, columns=["Price ($)", "Macaulay duration",
                                      "Modified duration", "DV01/PVBP ($/bp)", "Convexity"])
bond_risk = pd.concat([bonds, metrics], axis=1)
bond_risk.round(4)
```

	Bond	Face value ($mm)	Coupon rate	Maturity (years)	Yield	Price ($)	Macaulay
0	2-year Treasury	12.0	0.0400	2	0.0410	1.1977e+07	1.9419
1	5-year agency	10.0	0.0475	5	0.0485	9.9561e+06	4.5077
2	10-year corporate	8.0	0.0575	10	0.0610	7.7927e+06	7.7068

2 Exact Repricing Versus Approximations

For a yield shock Δy,

$$\frac{\Delta P}{P} \approx -D_{mod}\Delta y$$

and the duration-plus-convexity approximation is

$$\frac{\Delta P}{P} \approx -D_{mod}\Delta y + \frac{1}{2}C(\Delta y)^2.$$

```
selected = bond_risk.iloc[2]
shocks_bp = np.array([-150, -75, -25, 25, 75, 150])
base_price = selected["Price ($)"]
exact_prices = np.array([
    bond_metrics(selected["Face value ($mm)"] * 1_000_000,
                 selected["Coupon rate"], selected["Maturity (years)"],
                 selected["Yield"] + shock / 10_000, frequency)[0]
    for shock in shocks_bp
])
dy = shocks_bp / 10_000
duration_change = -selected["Modified duration"] * dy
duration_convexity_change = duration_change + 0.5 * selected["Convexity"] * dy**2
exact_change = exact_prices / base_price - 1

comparison = pd.DataFrame({
    "Yield shock (bp)": shocks_bp,
    "Duration-only change (%)": 100 * duration_change,
    "Duration + convexity change (%)": 100 * duration_convexity_change,
    "Exact price change (%)": 100 * exact_change,
    "Duration-only error (bp of price)": 10_000 * (duration_change - exact_change),
    "Duration + convexity error (bp of price)": 10_000 * (duration_convexity_change - exact_
})
comparison.round(3)
```

	Yield shock (bp)	Duration-only change (%)	Duration + convexity change (%)	Exact price change (%)
0	-150	11.218	11.998	12.039
1	-75	5.609	5.804	5.809
2	-25	1.870	1.891	1.892
3	25	-1.870	-1.848	-1.848
4	75	-5.609	-5.414	-5.419
5	150	-11.218	-10.439	-10.477

```
fig, ax = plt.subplots(figsize=(7.4, 4.0))
ax.plot(shocks_bp, 100 * exact_change, "o-", label="Exact repricing")
ax.plot(shocks_bp, 100 * duration_change, "s--", label="Duration only")
ax.plot(shocks_bp, 100 * duration_convexity_change, "^--", label="Duration + convexity")
ax.set(xlabel="Yield shock (bp)", ylabel="Price change (%)",
       title="10-year corporate bond repricing")
ax.grid(alpha=0.3)
ax.legend()
```

```
plt.tight_layout()
plt.show()
```

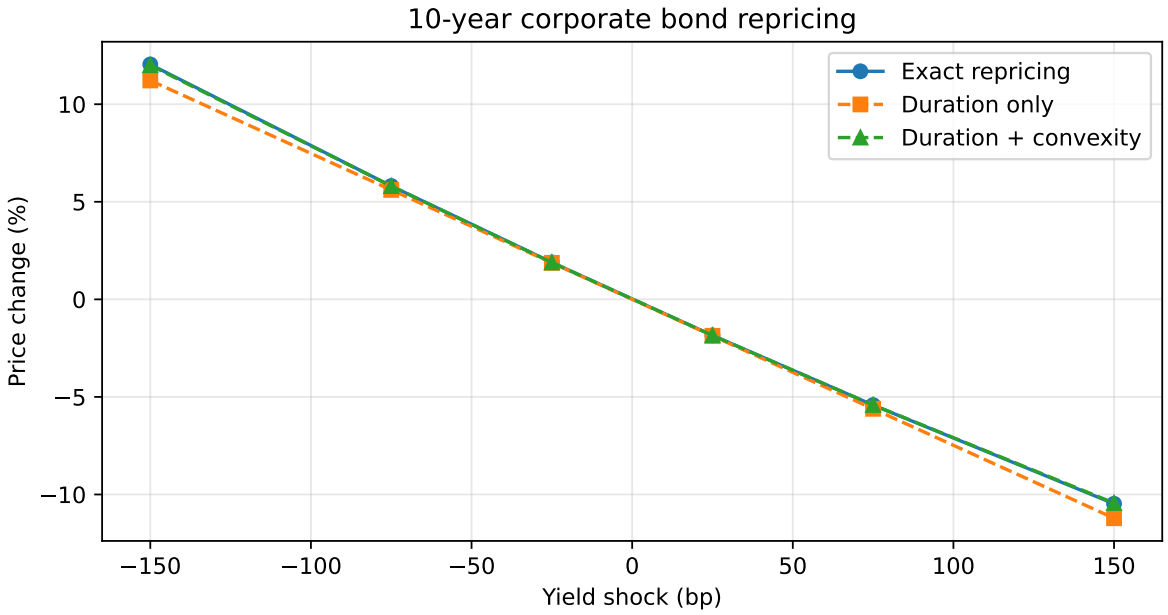

Figure 1: Convexity materially improves the approximation for larger yield moves

2.1 Explore the shock size (HTML)

The table and figure above are the static fallback in PDF. In HTML, move the slider to reprice the same 10-year 5.75% coupon bond exactly.

3 Portfolio Aggregation

Dollar risk aggregates across positions. Portfolio DV01 is the sum of bond DV01s; market-value-weighted duration is equivalent to total dollar duration divided by portfolio value.

```
portfolio_value = bond_risk["Price ($)"].sum()
portfolio_dv01 = bond_risk["DV01/PVBP ($/bp)"].sum()
portfolio_modified_duration = portfolio_dv01 / (portfolio_value * 0.0001)
portfolio_convexity = np.average(bond_risk["Convexity"], weights=bond_risk["Price ($)"])

pd.Series({
```

```
    "Portfolio market value ($)": portfolio_value,
    "Portfolio DV01/PVBP ($/bp)": portfolio_dv01,
    "Portfolio modified duration": portfolio_modified_duration,
    "Market-value-weighted convexity": portfolio_convexity,
}).round(3)
```

```
Portfolio market value ($)          2.9726e+07
Portfolio DV01/PVBP ($/bp)          1.2489e+04
Portfolio modified duration         4.2010e+00
Market-value-weighted convexity     2.7662e+01
dtype: float64
```

4 Parallel and Nonparallel Scenarios

A single duration number is designed for a parallel shift. A curve twist must be applied bond by bond.

```
scenario_shocks = {
    "Parallel +50 bp": np.array([50, 50, 50]),
    "Bear steepener": np.array([10, 35, 75]),
    "Bull flattener": np.array([-15, -45, -90]),
}

scenario_rows = []
for name, shocks in scenario_shocks.items():
    exact_pnl = 0.0
    duration_pnl = 0.0
    duration_convexity_pnl = 0.0
    for i, bond in bond_risk.iterrows():
        shock = shocks[i] / 10_000
        shocked_price = bond_metrics(
            bond["Face value ($mm)"] * 1_000_000, bond["Coupon rate"],
            bond["Maturity (years)"], bond["Yield"] + shock, frequency
        )[0]
        exact_pnl += shocked_price - bond["Price ($)"]
        duration_pnl += bond["Price ($)"] * (-bond["Modified duration"] * shock)
        duration_convexity_pnl += bond["Price ($)"] * (
            -bond["Modified duration"] * shock + 0.5 * bond["Convexity"] * shock**2
        )
    scenario_rows.append([name, duration_pnl, duration_convexity_pnl, exact_pnl])
```

```
scenario_results = pd.DataFrame(
    scenario_rows,
    columns=["Scenario", "Duration P&L ($)", "Duration + convexity P&L ($)", "Exact P&L ($)"]
).set_index("Scenario")
scenario_results.round(0)
```

Scenario	Duration P&L ($)	Duration + convexity P&L ($)	Exact P&L ($)
Parallel +50 bp	-624435.0	-614157.0	-614300.0
Bear steepener	-613245.0	-596639.0	-597027.0
Bull flattener	755876.0	780107.0	780811.0

The parallel scenario can be summarized by portfolio duration. The steepener and flattener require maturity-specific exposures; the same portfolio DV01 can hide materially different curve risk.

5 Validation Checks

```
assert np.all(bond_risk["Macaulay duration"] > bond_risk["Modified duration"])
assert np.all(bond_risk["DV01/PVBP ($/bp)"] > 0)
assert abs(portfolio_dv01 - bond_risk["DV01/PVBP ($/bp)"].sum()) < 1e-10
assert np.mean(np.abs(duration_convexity_change - exact_change)) < np.mean(np.abs(duration_ch
assert np.all(np.diff(exact_prices) < 0)

pd.Series({
    "Macaulay duration exceeds modified duration": True,
    "DV01 values are positive": True,
    "Portfolio DV01 equals component sum": True,
    "Convexity improves average approximation": True,
    "Exact price falls as yield rises": True,
})
```

```
Macaulay duration exceeds modified duration    True
DV01 values are positive                       True
Portfolio DV01 equals component sum            True
Convexity improves average approximation       True
Exact price falls as yield rises               True
dtype: bool
```

6 Financial Interpretation

Duration is a local slope, convexity is a curvature correction, and exact cash-flow repricing is the benchmark. DV01 translates percentage sensitivity into dollars. Portfolio aggregation is reliable for a stated parallel shock, but nonparallel scenarios require maturity-specific or key-rate exposures.